

GIT

Criar a pasta do projeto.

```
git init
git config user.name
git config user.email
git add .
{
se quiser cancelar antes do commit os arquivos
git rm --cached -r .
}
```

```
git commit -m "Primeiro commit"
git branch -M main
git remote add origin URL
git push -u origin main
```

Simulando branch's

Simular Aluno CSS

no CMD

```
cd C:\Users\gil_m\Downloads
```

```
git clone https://github.com/profgilmarfreire/eutealugo.git eutealugo-estilo
```

```
cd eutealugo-estilo
```

0. Entrar no VSCode

```
code .
```

1. Criar e entrar na branch

```
git switch -c estilo
```

2. Verificar se está na branch correta

```
git branch
```

3. Enviar a branch para o GitHub

Primeira vez:

```
git push -u origin estilo
```

O **-u** cria o vínculo entre a branch local e a branch remota.

Observe que depois do primeiro **push -u**, basta:

```
git push
```

5. Voltar para a branch principal

```
git switch main
```

Branch em maquinas diferentes

Simular Aluno CSS em outra maquina

no CMD

```
cd C:\Users\gil_m\Downloads
```

```
git clone https://github.com/profgilmarfreire/eutealugo.git eutealugo-estilo
```

```
cd eutealugo-estilo
```

0. Entrar no VSCode

```
code .
```

1. Criar e entrar na branch

```
git switch -c estilo
```

2. Verificar se está na branch correta

```
git branch
```

3. Configurar ambiente

```
git config user.name
```

```
git config user.email
```

4. Enviar arquivos para commit

```
git add .
```

```
{
```

```
se quiser cancelar antes do commit os arquivos
```

```
git rm --cached -r .
```

```
}
```

5. Commit

```
git commit -m "Primeiro commit"
```

6. Configuração do servidor remoto

Primeira vez:

```
git push -u origin estilo
```

O **-u** cria o vínculo entre a branch local e a branch remota.

Observe que depois do primeiro **push -u**, basta:

git push

7. Voltar para a branch principal

git switch main

Guia completo entre alunos

Git e GitHub - Trabalho em Equipe com Branches

Cenário

Projeto hospedado no GitHub:

eutealugo

Equipe:

- Aluno A → CSS / Estilização
- Aluno B → JavaScript

Cada aluno trabalhará em sua própria branch.

1. Clonar o Projeto

```
cd C:\Users\gil_m\Downloads
```

```
git clone https://github.com/profgilmarfreire/eutealugo.git eutealugo-estilo
```

```
cd eutealugo-estilo
```

2. Abrir no VS Code

```
code .
```

3. Verificar Configuração do Git

Verificar se o Git já possui usuário configurado:

```
git config --global user.name
```

```
git config --global user.email
```

Se não estiver configurado:

```
git config --global user.name "Nome do Aluno"
```

```
git config --global user.email "aluno@email.com"
```

4. Atualizar Projeto

Antes de iniciar o trabalho:

```
git pull
```

5. Criar e Entrar na Branch

Aluno CSS:

```
git switch -c estilo
```

Aluno JavaScript:

```
git switch -c javascript
```

6. Confirmar Branch Atual

```
git branch
```

Exemplo:

```
* estilo  
main
```

O asterisco (*) indica a branch atual.

7. Realizar Alterações

Editar arquivos CSS, HTML ou JavaScript.

8. Consultar Alterações

Verificar o que foi modificado:

```
git status
```

9. Adicionar Arquivos ao Staging

```
git add .
```

Conferir:

```
git status
```

Arquivos deverão aparecer em:

```
Changes to be committed
```

10. Cancelar o git add (Antes do Commit)

Caso tenha adicionado arquivos errados:

```
git rm --cached -r .
```

Verificar:

```
git status
```

11. Criar Commit

`git commit -m "Cria estilização da página inicial"`

12. Ver Histórico de Commits

Histórico resumido:

`git log --oneline`

Exemplo:

8a7f3c1 Cria estilização da página inicial
4d9e2b7 Adiciona menu responsivo
1f3a8d2 Primeiro commit

Histórico completo:

`git log`

13. Publicar a Branch no GitHub

Primeira vez:

`git push -u origin estilo`

ou

`git push -u origin javascript`

O parâmetro:

`-u`

cria o vínculo entre:

Branch Local -> Branch Remota
estilo -> origin/estilo

14. Próximos Envios

Após o primeiro push:

git push

15. Atualizar Alterações do GitHub

git pull

16. Trocar de Branch

Voltar para a principal:

git switch main

Ir para a branch CSS:

git switch estilo

Ir para a branch JavaScript:

git switch javascript

COMANDOS DE CONSULTA

Verificar status do projeto:

git status

Ver branch atual:

git branch

Ver todas as branches:

git branch -a

Ver histórico resumido:

git log --oneline

Ver histórico completo:

git log

Ver diferenças antes do commit:

git diff

Ver diferenças que já estão no staging:

git diff --staged

Ver repositório remoto configurado:

git remote -v

Ver usuário configurado:

git config --global user.name

git config --global user.email

COMANDOS DE RECUPERAÇÃO

Cancelar git add:

`git rm --cached -r .`

Atualizar projeto:

`git pull`

Trocar para a branch principal:

`git switch main`

Verificar onde está:

`git status`

`git branch`

FLUXO RESUMIDO

`git clone URL`

`cd projeto`

`code .`

`git pull`

`git switch -c estilo`

`git add .`

`git commit -m "Minha alteração"`

`git push -u origin estilo`

Próximas alterações:

`git add .`

`git commit -m "Nova alteração"`

`git push`

FLUXO MENTAL PARA O ALUNO

Quando estiver perdido:

1. Onde estou?

`git branch`

2. O que mudou?

`git status`

3. O que já fiz?

`git log --oneline`

4. Para onde estou enviando?

`git remote -v`

Esses quatro comandos resolvem a maioria dos problemas de iniciantes.

Estrutura do projeto

Estrutura Profissional de Projeto com Vite

Objetivo

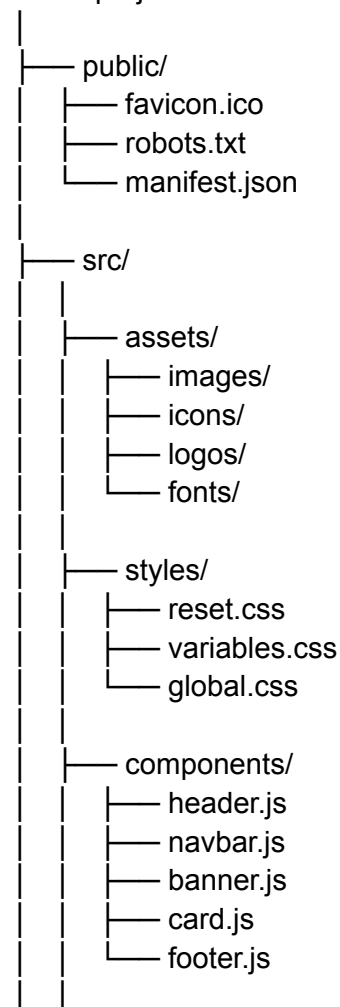
Organizar o projeto de forma profissional, utilizando uma arquitetura baseada em funcionalidades.

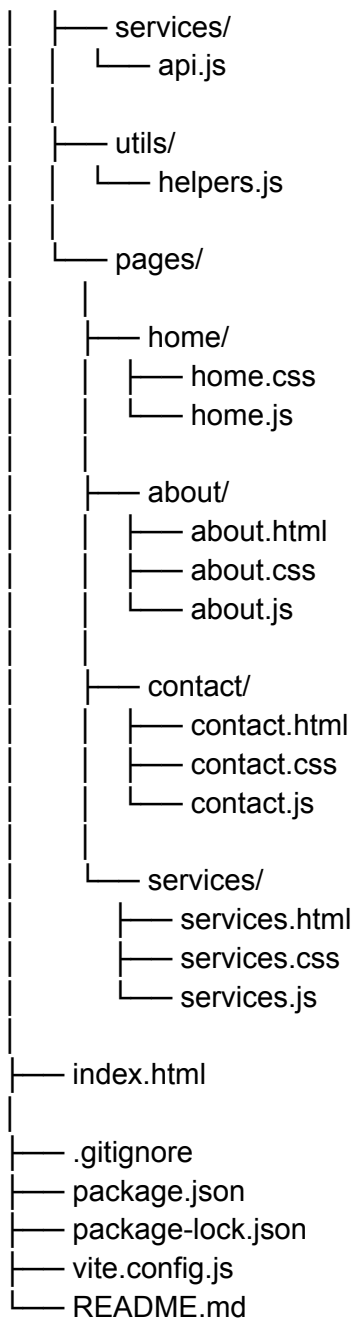
Cada página possui sua própria pasta contendo seus recursos específicos, enquanto os recursos compartilhados permanecem centralizados.

Essa organização facilita a manutenção, melhora a legibilidade do projeto e prepara o aluno para arquiteturas modernas utilizadas no mercado.

Estrutura do Projeto

nome-projeto/





Entendendo a Página Inicial (Home)

O arquivo principal da aplicação permanece na raiz do projeto:

index.html

Esse arquivo representa a página inicial do sistema.

Os recursos específicos da Home ficam organizados em:

```
pages/home/
├── home.css
└── home.js
```

Dessa forma:

- o HTML principal permanece facilmente acessível;
 - os arquivos da Home ficam agrupados;
 - a organização por funcionalidade é preservada;
 - a raiz do projeto permanece limpa.
-

Organização das Páginas

Cada funcionalidade possui sua própria pasta.

Home

```
home/
├── home.css
└── home.js
```

Arquivos exclusivos da página inicial.

About

```
about/
├── about.html
├── about.css
└── about.js
```

Arquivos exclusivos da página Sobre.

Contact

```
contact/
├── contact.html
├── contact.css
└── contact.js
```

Arquivos exclusivos da página Contato.

Services

```
services/  
├── services.html  
├── services.css  
└── services.js
```

Arquivos exclusivos da página Serviços.

Benefícios desta Arquitetura

- Raiz do projeto limpa.
 - Organização por funcionalidade.
 - Fácil manutenção.
 - Fácil localização dos arquivos.
 - Menor acoplamento.
 - Maior coesão.
 - Melhor legibilidade.
 - Reutilização de componentes.
 - Estrutura escalável.
 - Preparação para React, Vue e Angular.
-

Relação com Clean Code

Esta organização segue princípios importantes da engenharia de software:

- Separação de responsabilidades;
- Alta coesão;
- Baixo acoplamento;
- Organização lógica dos arquivos;
- Facilidade de manutenção;
- Facilidade de evolução do sistema.

O objetivo não é apenas criar aplicações que funcionem, mas desenvolver aplicações organizadas, compreensíveis e preparadas para crescer ao longo do tempo.

Estrutura Profissional de Projeto com Vite

Estrutura Profissional de Projeto com Vite

Capítulo 2 – Arquivos e Configurações do Projeto

Após compreender a estrutura de pastas, é importante entender a função de cada arquivo presente no projeto.

Muitos desses arquivos são criados automaticamente pelo Vite e pelo NPM, mas todos possuem uma finalidade específica.

public/

A pasta public armazena arquivos públicos.

Esses arquivos são disponibilizados diretamente para o navegador e não passam pelo processamento do Vite.

Exemplo:

```
public/  
├── favicon.ico  
├── robots.txt  
└── manifest.json
```

favicon.ico

É o ícone do site.

Ele aparece:

- na aba do navegador;
- nos favoritos;
- no histórico;

- nos atalhos criados pelo usuário.

Exemplo:



Sem o favicon, o navegador utiliza um ícone padrão.

robots.txt

Arquivo utilizado pelos mecanismos de busca.

Seu objetivo é informar quais áreas do site podem ou não ser indexadas.

Exemplo:

```
User-agent: *  
Disallow: /admin/
```

Significado:

- User-agent: todos os robôs;
- Disallow: não indexar a pasta admin.

Motores de busca como:

- Google
- Bing
- Yahoo

utilizam esse arquivo durante o rastreamento do site.

manifest.json

Arquivo utilizado principalmente em aplicações PWA (Progressive Web App).

Define informações da aplicação.

Exemplo:

```
{
```

```
"name": "Meu Sistema",
"short_name": "Sistema",
"start_url": "/",
"display": "standalone"
}
```

Permite:

- instalar o site como aplicativo;
- definir nome da aplicação;
- definir ícones;
- definir comportamento da abertura.

src/utils/

A pasta utils significa Utilities.

Em português:

Utilitários.

Ela armazena funções auxiliares reutilizáveis.

Exemplo:

```
utils/
└── helpers.js
```

helpers.js

Arquivo responsável por armazenar funções genéricas que podem ser utilizadas em qualquer lugar da aplicação.

Exemplo:

```
function formatarData(data){
  return data.toLocaleDateString();
}
```

Outro exemplo:

```
function validarEmail(email){  
  return email.includes("@");  
}
```

Benefícios:

- reutilização de código;
 - redução de duplicação;
 - manutenção simplificada.
-

src/services/

Responsável pela comunicação externa da aplicação.

Exemplo:

```
services/  
└── api.js
```

api.js

Centraliza as chamadas para APIs.

Exemplo:

```
async function buscarUsuarios(){  
  const resposta = await fetch("/usuarios");  
  return resposta.json();  
}
```

Benefícios:

- organização;
 - reaproveitamento;
 - facilidade de manutenção.
-

package.json

É o arquivo mais importante de um projeto Node.js.

Ele funciona como a identidade do projeto.

Exemplo:

```
{  
  "name": "meu-projeto",  
  "version": "1.0.0"  
}
```

Contém:

- nome do projeto;
 - versão;
 - dependências;
 - scripts;
 - informações gerais.
-

Dependências

Exemplo:

```
"dependencies": {  
  "vite": "^7.0.0"  
}
```

Significa que o projeto depende do Vite para funcionar.

Scripts

Exemplo:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build"  
}
```

Permite executar comandos como:

npm run dev
npm run build

package-lock.json

Arquivo gerado automaticamente pelo NPM.

Seu objetivo é registrar exatamente quais versões foram instaladas.

Exemplo:

vite 7.0.2

Mesmo que uma versão mais nova seja lançada posteriormente, todos os desenvolvedores utilizarão exatamente a mesma versão registrada.

Benefícios:

- previsibilidade;
- estabilidade;
- padronização do ambiente.

Normalmente não deve ser editado manualmente.

.gitignore

Arquivo utilizado pelo Git.

Define quais arquivos não devem ser enviados ao repositório.

Exemplo:

node_modules/
.env
dist/

Significado:

- node_modules → dependências instaladas;
- .env → variáveis sensíveis;
- dist → arquivos gerados automaticamente.

Benefício:

Evita enviar arquivos desnecessários para o GitHub.

vite.config.js

Arquivo de configuração do Vite.

Permite personalizar o comportamento da ferramenta.

Exemplo:

```
import { defineConfig } from 'vite'

export default defineConfig({
})
```

Pode ser utilizado para:

- configurar aliases;
 - configurar múltiplas páginas;
 - configurar plugins;
 - alterar diretórios de saída;
 - otimizar builds.
-

README.md

Arquivo de documentação do projeto.

Normalmente é a primeira coisa visualizada ao acessar um repositório no GitHub.

Exemplo:

Sistema de Vendas

Projeto desenvolvido utilizando Vite.

O README deve conter:

- descrição do projeto;
- tecnologias utilizadas;

- instruções de instalação;
 - instruções de execução;
 - autor do projeto.
-

node_modules/

Pasta criada automaticamente pelo NPM.

Contém todas as bibliotecas instaladas.

Exemplo:

node_modules/

Características:

- pode conter milhares de arquivos;
- não deve ser editada manualmente;
- normalmente não é enviada para o GitHub.

Caso seja apagada:

npm install

recria toda a pasta.

dist/

Pasta criada após o build da aplicação.

Exemplo:

npm run build

Resultado:

dist/

Ela contém os arquivos finais prontos para publicação.

Exemplo:

- HTML otimizado;
- CSS minificado;
- JavaScript minificado;
- imagens processadas.

É a pasta enviada para hospedagem.

Resumo Geral

Arquivo/Pasta	Função
favicon.ico	Ícone do site
robots.txt	Controle de indexação dos buscadores
manifest.json	Configuração de PWA
helpers.js	Funções auxiliares reutilizáveis
api.js	Comunicação com APIs
package.json	Configuração principal do projeto
package-lock.json	Controle de versões das dependências
.gitignore	Arquivos ignorados pelo Git
vite.config.js	Configuração do Vite
README.md	Documentação do projeto
node_modules	Dependências instaladas
dist	Arquivos finais para produção

Compreender esses arquivos é tão importante quanto compreender HTML, CSS e JavaScript, pois eles fazem parte da estrutura profissional utilizada em projetos modernos.